BiteSwipe

Database Product Document (Draft)

CS 514 Team Project
Spencer Barrett

October 2026

Contents

1 Introduction **2**

2 System architecture **2**

3 Database design **2**

3.1 Entities 2

3.2 ER diagram 3

3.3 Relationships 4

3.4 Relational schema 4

3.5 Normalization 5

4 Implementation **5**

4.1 Getting the data in 5

4.2 API 6

4.3 The feed query 6

4.4 Saving a swipe 7

5 Course concepts applied **7**

6 Screenshots **8**

7 User guide **8**

8 Limitations and next steps **9**

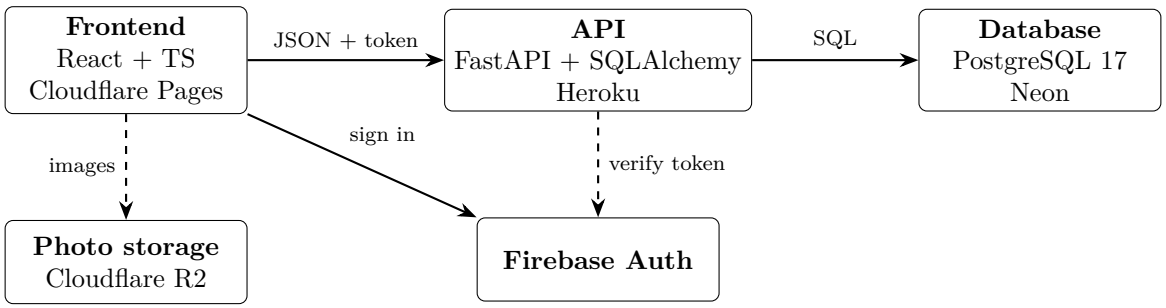
1 Introduction

BiteSwipe is a web app that helps you decide where to eat. It shows one restaurant at a time, and you like it if you'd go there or pass if you wouldn't. Every swipe gets saved, and the feed only shows restaurants you haven't swiped on yet.

We picked this project because the app is basically a thin layer over a relational database. What to show next, which places are open and what someone liked are all queries, so it was a good fit for what we covered in the course.

2 System architecture

The app has three layers. The React frontend talks to a FastAPI backend over HTTPS, and the backend is the only thing that talks to Postgres. Login goes through Firebase. The frontend gets a token from Firebase and sends it with every request. The backend checks that token on every route that touches a user's own data (the feed, swipes and profile). The restaurant lookup routes are public.



Layer	Tech	Hosted on
Frontend	React 19, TypeScript, Vite, Tailwind, shadcn/ui	Cloudflare Pages
Backend API	Python, FastAPI, SQLAlchemy, Alembic, uv	Heroku (Docker)
Database	PostgreSQL 17	Neon
Auth	Firebase Authentication (email + password)	Google
Images	Object storage	Cloudflare R2

We started out on Firestore because it was quick to set up with Firebase, and switched to Postgres early on. Our data is relational (restaurants have many cuisines, users swipe on many restaurants), and with Firestore we were checking those relationships in application code. Postgres enforces them with foreign keys and constraints.

3 Database design

3.1 Entities

The schema has seven tables:

- **users**: one row per person who has logged in. We store their Firebase UID, not a password.
- **restaurants**: the core entity, with name, address, coordinates, price level and Google's rating.
- **cuisines**: a lookup table of cuisine names (97 of them).
- **restaurant_cuisines**: a junction table that resolves the many-to-many between restaurants and cuisines.
- **restaurant_photos**: photos per restaurant, with a `position` column so position 0 is the main card image.
- **restaurant_hours**: one row per opening interval per weekday.
- **swipes**: one row per user per restaurant, saying whether they liked it.

3.2 ER diagram



Figure 1: The Entity-Relationship Diagram.

3.3 Relationships

Relationship	Cardinality	What it means
users – restaurants (via swipes)	M:N	A user swipes many restaurants, and a restaurant gets swiped by many users.
restaurants – cuisines (via restaurant_cuisines)	M:N	A taco shop can be "mexican" and "seafood", and "mexican" covers lots of places.
restaurants – restaurant_photos	1:N	546 of our 550 restaurants have at least one photo.
restaurants – restaurant_hours	1:N	No row for a day means the place is closed that day.

3.4 Relational schema

```
users(id, firebase_uid, email, display_name, created_at)
  PK (id), UNIQUE (firebase_uid)

restaurants(id, name, address_line1, city, state, zip, lat, lng,
           price_level, external_rating, external_rating_count,
           phone, website, google_place_id, source_id, created_at)
  PK (id)

cuisines(id, name)
  PK (id), UNIQUE (name)

restaurant_cuisines(restaurant_id, cuisine_id)
  PK (restaurant_id, cuisine_id)
  FK restaurant_id -> restaurants(id) ON DELETE CASCADE
  FK cuisine_id -> cuisines(id) ON DELETE CASCADE

restaurant_photos(id, restaurant_id, photo_ref, position, url)
  PK (id), UNIQUE (restaurant_id, position)
  FK restaurant_id -> restaurants(id) ON DELETE CASCADE

restaurant_hours(id, restaurant_id, day_of_week, open_time,
                close_time, closes_next_day)
  PK (id)
  FK restaurant_id -> restaurants(id) ON DELETE CASCADE

swipes(id, user_id, restaurant_id, liked, created_at)
  PK (id), UNIQUE (user_id, restaurant_id)
  FK user_id -> users(id) ON DELETE CASCADE
  FK restaurant_id -> restaurants(id) ON DELETE CASCADE
```

We define the tables as SQLAlchemy models. Alembic turns those models into migrations that create the tables. The Python models are the source of truth, and the schema above comes from them.

3.5 Normalization

Our raw data was a JSON export shaped for Firestore, with arrays nested inside each record and fields computed from other fields. We normalized it to 3NF:

- **1NF.** Each restaurant had a `cuisines` array and a `photoRefs` array inside it. We pulled those out into `restaurant_cuisines` and `restaurant_photos`, so every cell holds a single value. Hours were stored as strings like "Mon–Fri 11am–9pm", so they were parsed into one row per interval in `restaurant_hours`.
- **2NF.** The only composite key ise `restaurant_cuisines(restaurant_id, cuisine_id)`, and it has no other columns.
- **3NF.** The export had a `geohash` field computed from `lat/lng`. That's a derived value so we dropped it. Postgres can index coordinates directly. The export also had a `stats` object with like/pass counts. Those can be computed from `swipes` with a `COUNT`, so we didn't store them either.

4 Implementation

4.1 Getting the data in

The restaurant data came from a raw export of 560 San Diego restaurants. We wrote a separate Python cleaning pipeline that:

1. filled in ratings, price level and photo references from the Google Places API, with a hard cap on API calls so we stayed in the free tier;
2. parsed the messy hours strings into structured day/open/close rows, and dropped the six restaurants whose hours we couldn't parse (554 left);
3. dropped four more whose coordinates were nowhere near San Diego (latitudes around 34–36°), leaving 550;
4. split cuisines out into their own table and removed duplicate names;
5. wrote one clean CSV per table, plus a report of every issue it found.

A seed script (`biteswipe_api.scripts.seed`) loads the CSVs in chunks of 500 rows. The CSVs carry their own IDs so the foreign keys line up, which means the script has to call `setval` afterwards to move each table's sequence past the highest ID.

Table	Rows loaded
restaurants	550
cuisines	97
restaurant_cuisines	1,086
restaurant_photos	1,633
restaurant_hours	3,690

Photos are downloaded once and stored in Cloudflare R2. The `url` column stays null until a photo has been uploaded, so the app never shows a broken image.

4.2 API

Method	Route	Auth	What it does
GET	`/api/me`	yes	Returns the logged-in user, creating their row on first login.
GET	`/api/feed`	yes	The next batch (default 10) of restaurants the user hasn't swiped. Optional `cuisine` filter.
POST	`/api/swipes`	yes	Saves a like or pass, or changes an existing one.
GET	`/api/swipes`	yes	The user's swipe history, newest first. Optional `liked` filter.
GET	`/api/restaurants`	no	Paged list of restaurants, with an optional `cuisine` filter.
GET	`/api/restaurants/nearby`	no	Restaurants within `radius_mi` (default 3, max 30) of a lat/lng, closest first.
GET	`/api/restaurants/{id}`	no	One restaurant with phone, website, hours, photos and cuisines.
GET	`/api/health`	no	Health check.

A FastAPI dependency, `get_current_user`, verifies the Firebase token and looks up the user by `firebase_uid`. If there's no row yet, it inserts one with `ON CONFLICT (firebase_uid) DO NOTHING`. That way, if two requests from a brand-new user arrive at the same time, they can't create two rows.

4.3 The feed query

This is the main query in the app. It finds restaurants the user hasn't swiped on, best-rated first. We write it with the SQLAlchemy query builder, and this is the SQL it sends:

```
SELECT restaurants.*
FROM restaurants
WHERE NOT (EXISTS (
  SELECT 1 FROM swipes
  WHERE swipes.user_id = :user_id
    AND swipes.restaurant_id = restaurants.id
))
ORDER BY restaurants.external_rating DESC NULLS LAST, restaurants.id
LIMIT 10;
```

The `NOT EXISTS` keeps each restaurant that has no matching swipe. The unique index on `swipes (user_id, restaurant_id)` makes that check fast. `NULLS LAST` keeps the four restaurants without a Google rating from ending up at the top.

Each card also needs its cuisines and photos. We use SQLAlchemy's `selectinload`, which grabs them in one query per relationship:

```
SELECT ... FROM restaurant_cuisines
JOIN cuisines ON cuisines.id = restaurant_cuisines.cuisine_id
WHERE restaurant_cuisines.restaurant_id IN (:id_1, ..., :id_10);

SELECT ... FROM restaurant_photos
WHERE restaurant_photos.restaurant_id IN (:id_1, ..., :id_10)
ORDER BY restaurant_photos.position;
```

4.4 Saving a swipe

Because of the unique constraint, saving a swipe is an upsert:

```
INSERT INTO swipes (user_id, restaurant_id, liked)
VALUES (:user_id, :restaurant_id, :liked)
ON CONFLICT (user_id, restaurant_id)
DO UPDATE SET liked = :liked
RETURNING *;
```

If the restaurant ID doesn't exist, the foreign key fails, and the API rolls back and returns a 404.

On the frontend, the card leaves the deck right away and the request runs in the background (an optimistic update). When three or fewer cards are left, the deck fetches more and drops any IDs it already has or has already swiped. We added that filter after a bug where a refill overlapping a swipe showed the same restaurant twice.

5 Course concepts applied

Concept	Where it shows up
ER modeling	Seven entities, two M:N relationships resolved with junction tables (§3.2).
Normalization (1NF–3NF)	Splitting out arrays, dropping `geohash` and the stored like/pass counts (§3.5).
Keys and integrity	Surrogate PKs, composite PK on `restaurant_cuisines`, FKs with `ON DELETE CASCADE`.
Constraints	`UNIQUE` on swipes and photo position, `CHECK` on price, rating, coordinates and day.
Joins	Cuisines through the junction table; swipe history joins `swipes` to `restaurants`.
Subqueries / anti-join	`NOT EXISTS` in the feed; `EXISTS` for the cuisine filter.
Query efficiency	`selectinload` batching to avoid N+1
Sorting / pagination	`ORDER BY ... NULLS LAST`, `LIMIT`/`OFFSET` on list routes.
Indexing	Composite, unique and partial indexes.
Upserts / transactions	`ON CONFLICT DO UPDATE` for swipes, `DO NOTHING` for new users, rollback on FK failure.
Migrations	Schema versioned with Alembic (two migrations so far).
Data cleaning / ETL	Raw JSON to clean CSVs, loaded into Postgres (§4.1).

6 Screenshots

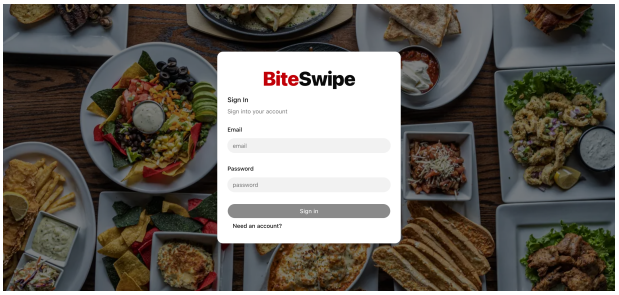


Figure 2: The login screen.

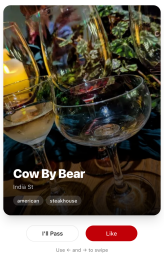


Figure 3: The card deck.

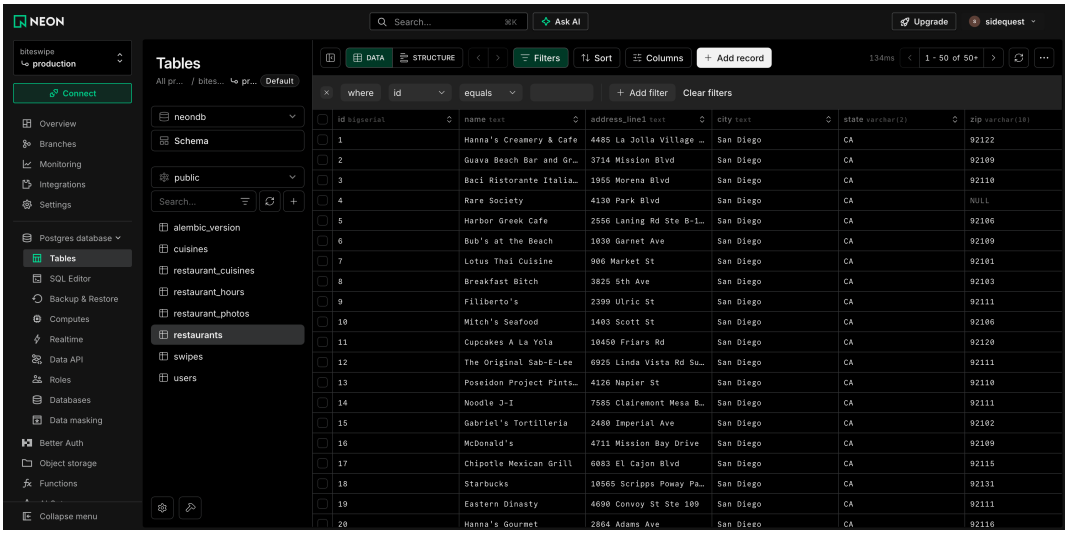


Figure 4: Neon Database.

7 User guide

The live app is at `https://biteswipe-app.pages.dev/`. It runs in any browser.

1. Register or sign in with an email and password. To make a new account, click *Need an account?*.
2. Each card shows a photo, the name, the address and up to four cuisine tags. Click **Like** or press → if you'd go there, or click **I'll Pass** or press ← to skip it.
3. New cards load automatically, and you won't see the same restaurant twice. When you've seen them all, you'll get a "That's everywhere" screen.
4. Click **Log out** at the top when you're done. Your swipes stay saved to your account.

To run it locally, clone `https://github.com/spencer-barrett/CS514-BiteSwipe` and follow the README in `apps/backend/api`. In short: start Postgres with `docker compose up -d`, run `uv`

`run alembic upgrade head` and the seed script, start the API with `uvicorn`, then run `npm run dev` in `apps/biteswipe-app`.

8 Limitations and next steps

- The data is a one-time snapshot of 550 San Diego restaurants, so new places and updated ratings won't show up until we reload it.
- You can only swipe with buttons or the arrow keys, not touch gestures. The card doesn't show rating or price yet, and there's no screen for the restaurant detail or swipe-history routes, even though the API supports both.
- The cleaning report flagged 25 opening intervals like 11:00 to 09:00 that are probably typos for 9pm, not real overnight hours. We kept them and haven't fixed them yet.
- The feed only sorts by Google's rating. Next we want to weight restaurants by the cuisines a user tends to like, using what's already in `swipes`.
- Because `google_place_id` can't be unique, nothing stops the same business from being loaded twice. A `UNIQUE (name, address_line1)` constraint would fix that once we've confirmed the data is clean.